PRINCEDEB / P000415SE-SCOUT-School-Safety-Dashboard-Prototype-

Type / to search

Code Issues Pull requests 2 Actions Projects Wiki Security and quality Insights Settings

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: murali/backend-v3 ✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests

Create pull request

1 commit | 10 files changed | 1 contributor

Commits on Apr 24, 2026

feat: add analytics cache + remove polling to reduce Firebase reads

MuraliGMD committed 3 minutes ago ✓

89c8143

Showing 10 changed files with **314 additions** and **353 deletions**.

Split **Unified**

6 client/src/components/Layout.jsx

```
3  3
4  4  export default function Layout({ children }) {
5  5    const location = useLocation()
```

```
6      -  const { currentUser, logout, isAdmin } = useAuth()
       6  const { currentUser, userRole, logout, isAdmin } = useAuth()
7      7
8      8  if (!currentUser) {
9      9    return <Navigate to="/login" replace />
65     65       <div className="p-3 border-t border-gray-200">
66     66         <div className="px-3 py-2 mb-1">
67     67           <div className="flex items-center justify-between mb-0.5">
68     -              <p className="text-sm font-medium text-gray-800">{currentUser.name}</p>
       68             <p className="text-sm font-medium text-gray-800">{currentUser.displayName || currentUser.name}</p>
69     69             <span className={`text-xs px-2 py-0.5 rounded-full font-medium ${
70     70               isAdmin ? 'bg-red-100 text-red-700' : 'bg-blue-100 text-blue-700'
71     71             }`}>
72     -              {currentUser.role}
       72             {userRole || currentUser.role}
73     73             </span>
74     74           </div>
75     75           <p className="text-xs text-gray-400">{currentUser.email}</p>
```

33 client/src/context/AuthContext.jsx

```
2      2
3      3  const AuthContext = createContext(null)
4      4
5      -  const mockUsers = [
6      -    { id: '1', name: 'Admin User', email: 'admin@school.edu', password: 'password123', role: 'admin' },
7      -    { id: '2', name: 'Staff User', email: 'user@school.edu', password: 'password123', role: 'user' },
8      -  ]
9      -
10     5  export function AuthProvider({ children }) {
11     6    const [currentUser, setCurrentUser] = useState(null)
       7  const [userRole, setUserRole] = useState(null)
12     8
13     -  const login = (email, password) => {
14     -    const found = mockUsers.find(u => u.email === email && u.password === password)
15     -    if (found) {
```

```
16  -       setCurrentUser(found)
17  -       return { success: true }
18  -     }
19  -     return { success: false, message: 'Invalid email or password' }
     9  +   // login(mockUser, role) — matches main branch AuthContext API.
    10  +   // mockUser is a plain object with uid, email, displayName, photoURL.
    11  +   // role is a string: 'Admin' | 'User'
    12  +   async function login(mockUser, role) {
    13  +     setCurrentUser(mockUser)
    14  +     setUserRole(role)
20  15      }
21  16
22  -   const logout = () => setCurrentUser(null)
    17  +   function logout() {
    18  +     setCurrentUser(null)
    19  +     setUserRole(null)
    20  +   }
23  21
24  22      return (
25  -     <AuthContext.Provider value={{ currentUser, login, logout, isAdmin: currentUser?.role === 'admin' }}>
    23  +     <AuthContext.Provider value={{
    24  +       currentUser,
    25  +       userRole,
    26  +       login,
    27  +       logout,
    28  +       isAdmin: userRole === 'Admin',
    29  +       isUser: userRole === 'User',
    30  +     }}>
26  31        {children}
27  32      </AuthContext.Provider>
28  33      )
```

30 client/src/data/mockData.js

```
...  ...  @@ -1,3 +1,33 @@
```

```
+ // ── MOCK_USERS — mirrors the real users collection in Firestore ──
+ // Emails and roles match exactly what's stored in the DB (role is lowercase).
+ // Roles are normalised to title-case in Login.jsx before being passed to AuthContext.
+ export const MOCK_USERS = [
+   {
+     uid: 'admin',
+     email: 'admin@school.edu',
+     password: 'password123',
+     displayName: 'Admin User',
+     role: 'admin',
+     photoURL: null,
+   },
+   {
+     uid: 'mock-user-001',
+     email: 'user@school.edu',
+     password: 'password123',
+     displayName: 'Staff User',
+     role: 'user',
+     photoURL: null,
+   },
+   {
+     uid: 'murali',
+     email: 'murali@school.edu',
+     password: 'password123',
+     displayName: 'Murali',
+     role: 'admin',
+     photoURL: null,
+   },
+ ]
+
  export const incidents = [
    {
      id: '1',
```

43 client/src/pages/Analytics.jsx

```diff
@@ -1,4 +1,5 @@
 import { useState, useEffect } from 'react'
+import { Navigate } from 'react-router-dom'
 import { useAuth } from '../context/AuthContext'
 import { analyticsAPI } from '../api/client'
 import {
      setLocationData(data.locationData)
      setIncidentsByDay(data.incidentsByDay)
      setResponseTimeData(data.responseTimeData)
-     return true  // success
    } catch (err) {
      setError(err.message || 'Failed to load analytics data')
-     return false  // failure — caller should stop polling
    } finally {
      setLoading(false)
    }
  }

-  // Fetch on mount, then poll every 30s — stop polling on any error
+  // Fetch once on mount — no polling (saves ~2,280 Firestore reads/hour)
  useEffect(() => {
    if (!isAdmin) return
-
-    let interval = null
-
-    fetchAnalytics().then(ok => {
-      if (ok) {
-        interval = setInterval(async () => {
-          const ok = await fetchAnalytics()
-          if (!ok) {
-            clearInterval(interval)
-            interval = null
-          }
-        }, 30000)
-      }
```

```
65  -     })
66  -
67  -     return () => { if (interval) clearInterval(interval) }
    51 +     fetchAnalytics()
68  52     }, [isAdmin])
69  53
    54 +   // RBAC guard — redirect non-admins (matches main branch pattern)
70  55     if (!isAdmin) {
71  -     return (
72  -       <div className="p-6 max-w-lg mx-auto mt-20 text-center">
73  -         <div className="bg-white border border-gray-200 rounded-xl p-10">
74  -           <div className="text-5xl mb-4">🔒</div>
75  -           <h2 className="text-xl font-bold text-gray-900 mb-2">Restricted Access</h2>
76  -           <p className="text-sm text-gray-500">
77  -             The Analytics page is only available to Admin users.
78  -             Contact your Safety Manager if you need access.
79  -           </p>
80  -         </div>
81  -       </div>
82  -     )
    56 +     return <Navigate to="/dashboard" replace />
83  57     }
84  58
85  59     if (loading) {
118 92           <h1 className="text-2xl font-bold text-gray-900">Analytics</h1>
119 93           <p className="text-sm text-gray-500">Operational insights</p>
120 94         </div>
    95 +         <button
    96 +           onClick={fetchAnalytics}
    97 +           disabled={loading}
    98 +           className="px-3 py-2 bg-blue-600 text-white text-sm rounded-lg hover:bg-blue-700 disabled:bg-gray-400
       transition-colors flex items-center gap-2"
    99 +         >
    100 +           {loading ? '⏳ Refreshing...' : '🔄 Refresh'}
    101 +         </button>
```

```
121  102        </div>
122  103
123  104        {/* Summary Cards */}
```

2 client/src/pages/Dashboard.jsx

```
71   71        <div className="flex items-center justify-between mb-6">
72   72          <div>
73   73            <h1 className="text-2xl font-bold text-gray-900">Dashboard</h1>
74   -            <p className="text-sm text-gray-500">Welcome back, {currentUser?.name || currentUser?.email || 'Admin'}</p>
     74 +         <p className="text-sm text-gray-500">Welcome back, {currentUser?.displayName || currentUser?.name || currentUser?.email || 'Admin'}</p>
75   75          </div>
76   76          <button
77   77            onClick={() => navigate('/submit')}
```

47 client/src/pages/Login.jsx

```
...  ...  @@ -1,12 +1,15 @@
1    1    import { useState } from 'react'
2    2    import { useNavigate } from 'react-router-dom'
3    3    import { useAuth } from '../context/AuthContext'
     4 +  import { MOCK_USERS } from '../data/mockData'
4    5
5    -  const demoAccounts = [
6    -    { name: 'Admin User', email: 'admin@school.edu', role: 'admin', color: 'bg-red-100 text-red-700' },
7    -    { name: 'Staff User', email: 'user@school.edu', role: 'user', color: 'bg-blue-100 text-blue-700' },
8    -    { name: 'Murali User', email: 'gmd@school.edu', role: 'user', color: 'bg-green-100 text-green-700' },
9    -  ]
     6 +  // Demo account quick-fill buttons — kept in sync with MOCK_USERS
     7 +  const demoAccounts = MOCK_USERS.map(u => ({
     8 +    name: u.displayName,
     9 +    email: u.email,
     10 +   role: u.role,
     11 +   color: u.role === 'admin' ? 'bg-red-100 text-red-700' : 'bg-blue-100 text-blue-700',
```

```
+ }))

  export default function Login() {
    const [email, setEmail] = useState('')
      setError('')
      setLoading(true)
      await new Promise(resolve => setTimeout(resolve, 400))
-     const result = login(email, password)
-     setLoading(false)

-     if (result.success) {
-       navigate('/dashboard')
+     // Match against MOCK_USERS (same source as main branch)
+     const matched = MOCK_USERS.find(
+       u => u.email.toLowerCase() === email.trim().toLowerCase() && u.password === password
+     )
+
+     if (!matched) {
+       setError('Invalid email or password.')
+       setLoading(false)
        return
      }

-     setError(result.message)
+     // Build the user object AuthContext expects (matches main branch shape)
+     const mockFirebaseUser = {
+       uid: matched.uid,
+       email: matched.email,
+       displayName: matched.displayName,
+       name: matched.displayName,
+       photoURL: matched.photoURL ?? null,
+     }
+
+     // Normalise role to title-case so isAdmin check ('Admin') works
+     // regardless of how the role is stored (Firestore uses lowercase 'admin')
```

```
+     const normalisedRole = matched.role.charAt(0).toUpperCase() + matched.role.slice(1)
+
+     try {
+       await login(mockFirebaseUser, normalisedRole)
+       navigate('/dashboard')
+     } catch (err) {
+       setError('Login failed: ' + err.message)
+     } finally {
+       setLoading(false)
+     }
33    }
34
35    return (
```

54 client/src/pages/Notifications.jsx

```
25    const [notifications, setNotifications] = useState([])
26    const [loading, setLoading] = useState(true)
27
28  - useEffect(() => {
29  -   const fetch = async () => {
30  -     try {
31  -       const data = await notificationsAPI.list()
32  -       setNotifications(data)
33  -     } catch (err) {
34  -       console.error('Failed to fetch notifications:', err)
35  -     } finally {
36  -       setLoading(false)
37  -     }
28  + const fetchNotifications = async () => {
29  +   setLoading(true)
30  +   try {
31  +     const data = await notificationsAPI.list()
32  +     setNotifications(data)
33  +   } catch (err) {
34  +     console.error('Failed to fetch notifications:', err)
```

```
+     } finally {
+       setLoading(false)
      }
-     fetch()
-     const interval = setInterval(fetch, 15000)
-     return () => clearInterval(interval)
+   }
+
+   // Fetch once on mount — no polling (saves ~24,000 Firestore reads/hour)
+   useEffect(() => {
+     fetchNotifications()
    }, [])

    // Admin sees all, user sees only their own
    const visibleNotifications = isAdmin
      ? notifications
-     : notifications.filter(n => n.recipientEmail === currentUser.email)
+     : notifications.filter(n => n.recipientEmail === currentUser?.email)

    const totalSent = visibleNotifications.filter(
      n => n.sms === 'sent' || n.email === 'sent'
      <div className="p-6 max-w-5xl mx-auto">

        {/* Header */}
-       <div className="mb-6">
-         <h1 className="text-2xl font-bold text-gray-900">Notifications</h1>
-         <p className="text-sm text-gray-500">
-           {isAdmin
-             ? 'Track SMS and email delivery across all incidents'
-             : 'Your personal notification history'}
-         </p>
+       <div className="mb-6 flex items-center justify-between">
+         <div>
+           <h1 className="text-2xl font-bold text-gray-900">Notifications</h1>
+           <p className="text-sm text-gray-500">
```

```
+            {isAdmin
+              ? 'Track SMS and email delivery across all incidents'
+              : 'Your personal notification history'}
+          </p>
+        </div>
+        <button
+          onClick={fetchNotifications}
+          disabled={loading}
+          className="px-3 py-2 bg-blue-600 text-white text-sm rounded-lg hover:bg-blue-700 disabled:bg-gray-400 transition-colors flex items-center gap-2"
+        >
+          {loading ? '⏳ Refreshing...' : '🔄 Refresh'}
+        </button>
        </div>

        {/* User info banner */}
                  <div className="flex-1 min-w-0">
                    <p className="text-sm font-medium text-gray-800">{n.incidentTitle}</p>
                    <p className="text-xs text-gray-500 mt-0.5">
-                     👤 {n.recipientName} · 📧 {n.recipientEmail} · 📱 {n.recipientPhone}
+                     👤 {n.recipient name} · 📧 {n.recipientEmail}
                    </p>
                    <p className="text-xs text-gray-400 mt-0.5">🕐 {n.timestamp}</p>
                  </div>
```

151 server/src/analyticsCache.js

```
@@ -0,0 +1,151 @@
+ // analyticsCache.js — In-memory analytics cache for SCOUT
+ // Eliminates repeated Firestore reads by caching the full analytics payload.
+ // Cache is invalidated whenever an incident is created or its status changes.
+
+ const { getDb } = require('./db/firebase')
+
+ // ── Cache state ──────────────────────────────────────────────────────────
```

```
+ let cachedData = null      // Holds the full analytics payload when warm
+ let cacheIsDirty = true    // true = needs rebuild from Firestore on next request
+
+ // ── Helpers ──────────────────────────────────────────────────────────────
+
+ function toDate(val) {
+   if (!val) return null
+   if (typeof val.toDate === 'function') return val.toDate()
+   const d = new Date(val)
+   return isNaN(d) ? null : d
+ }
+
+ function getResponseTime(incident) {
+   if (incident.status !== 'resolved') return null
+   const created = toDate(incident.createdAt)
+   const updated = toDate(incident.updatedAt)
+   if (!created || !updated) return null
+   const mins = (updated - created) / 60000
+   return mins > 0 ? mins : null
+ }
+
+ // ── Build analytics from Firestore ──────────────────────────────────────
+ async function buildAnalytics() {
+   const db = getDb()
+   const snapshot = await db.collection('incidents').get()
+   const incidents = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }))
+
+   const now = new Date()
+   const weekAgo = new Date(now.getTime() - 7 * 24 * 60 * 60 * 1000)
+
+   // Summary metrics
+   const totalIncidents = incidents.length
+   const resolvedCount = incidents.filter(i => i.status === 'resolved').length
+   const allResponseTimes = incidents.map(getResponseTime).filter(v => v !== null)
+   const avgResponseTime = allResponseTimes.length > 0
```

```
+     ? parseFloat((allResponseTimes.reduce((sum, v) => sum + v, 0) / allResponseTimes.length).toFixed(1))
+     : 0
+   const thisWeekIncidents = incidents.filter(i => {
+     const d = toDate(i.createdAt)
+     return d && d >= weekAgo
+   }).length
+
+   // Incidents by type
+   const typeCounts = {}
+   incidents.forEach(i => {
+     const type = i.type?.charAt(0).toUpperCase() + i.type?.slice(1) || 'Unknown'
+     typeCounts[type] = (typeCounts[type] || 0) + 1
+   })
+   const incidentsByType = Object.entries(typeCounts).map(([type, count]) => ({ type, count }))
+
+   // Status breakdown
+   const statusMap = { resolved: 'Resolved', acknowledged: 'Acknowledged', triggered: 'Triggered', archived: 'Archived' }
+   const statusCounts = {}
+   Object.values(statusMap).forEach(s => { statusCounts[s] = 0 })
+   incidents.forEach(i => {
+     const name = statusMap[i.status] || 'Unknown'
+     statusCounts[name] = (statusCounts[name] || 0) + 1
+   })
+   const statusBreakdown = Object.entries(statusCounts).map(([name, value]) => ({ name, value }))
+
+   // By location
+   const locationCounts = {}
+   incidents.forEach(i => {
+     const loc = i.location || 'Unknown'
+     locationCounts[loc] = (locationCounts[loc] || 0) + 1
+   })
+   const locationData = Object.entries(locationCounts).map(([location, count]) => ({ location, count }))
+
+   // This week by day (Mon-Sun)
```

```
+ const startOfWeek = new Date(now)
+ const dowOffset = (startOfWeek.getUTCDay() === 0 ? -6 : 1 - startOfWeek.getUTCDay())
+ startOfWeek.setUTCDate(startOfWeek.getUTCDate() + dowOffset)
+ startOfWeek.setUTCHours(0, 0, 0, 0)
+ const days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
+ const dayMap = { 1: 'Mon', 2: 'Tue', 3: 'Wed', 4: 'Thu', 5: 'Fri', 6: 'Sat', 0: 'Sun' }
+ const dayCounts = {}
+ days.forEach(d => { dayCounts[d] = 0 })
+ incidents.forEach(i => {
+   const d = toDate(i.createdAt)
+   if (d && d >= startOfWeek) dayCounts[dayMap[d.getUTCDay()]]++
+ })
+ const incidentsByDay = days.map(d => ({ day: d, incidents: dayCounts[d] }))
+
+ // Response time trend (last 4 weeks)
+ const weekData = {}
+ for (let i = 0; i < 4; i++) weekData[`Week ${i + 1}`] = { total: 0, count: 0 }
+ incidents.forEach(i => {
+   const d = toDate(i.createdAt)
+   if (!d) return
+   const weekNumber = Math.floor((now - d) / (7 * 24 * 60 * 60 * 1000)) + 1
+   if (weekNumber >= 1 && weekNumber <= 4) {
+     const key = `Week ${5 - weekNumber}`
+     if (weekData[key]) {
+       const rt = getResponseTime(i)
+       if (rt !== null) { weekData[key].total += rt; weekData[key].count++ }
+     }
+   }
+ })
+ const responseTimeData = Object.entries(weekData).map(([week, d]) => ({
+   week,
+   avgMinutes: d.count > 0 ? parseFloat((d.total / d.count).toFixed(1)) : 0,
+ }))
+
+ return {
```

```
+     summary: { totalIncidents, resolvedCount, avgResponseTime, thisWeekIncidents },
+     incidentsByType,
+     statusBreakdown,
+     locationData,
+     incidentsByDay,
+     responseTimeData,
+   }
+ }
+
+ // ── Public API ──────────────────────────────────────────────────────────
+
+ /**
+  * Returns the full analytics payload.
+  * Reads from Firestore only when the cache is dirty (first request, or after invalidation).
+  * All subsequent requests return the in-memory cache — 0 Firestore reads.
+  */
+ async function getAnalytics() {
+   if (!cacheIsDirty && cachedData) {
+     console.log('📊 Analytics served from cache (0 Firestore reads)')
+     return cachedData
+   }
+
+   console.log('🔄 Analytics cache miss — rebuilding from Firestore...')
+   cachedData = await buildAnalytics()
+   cacheIsDirty = false
+   console.log('✅ Analytics cache rebuilt')
+   return cachedData
+ }
+
+ /**
+  * Mark the cache as dirty so the next request rebuilds from Firestore.
+  * Call this whenever an incident is created or its status changes.
+  */
+ function invalidateAnalyticsCache() {
+   cacheIsDirty = true
```

```
+   cachedData = null
+   console.log('🗑  Analytics cache invalidated')
+ }
+
+ module.exports = { getAnalytics, invalidateAnalyticsCache }
```

298 server/src/routes/analytics.js

```
@@ -1,323 +1,77 @@
+ // analytics.js — All routes now served from in-memory cache.
+ // Firestore is only read on first request or after an incident changes.
+ // See analyticsCache.js for the cache logic and Firestore query details.
+
  const express = require('express')
- const { getDb, serverTimestamp } = require('../db/firebase')
+ const { getAnalytics } = require('../analyticsCache')

  const router = express.Router()

- // Helper: get Date from Firestore Timestamp or ISO string
- function toDate(val) {
-   if (!val) return null
-   if (typeof val.toDate === 'function') return val.toDate()
-   const d = new Date(val)
-   return isNaN(d) ? null : d
- }
-
- // Helper: calculate response time in minutes between createdAt and updatedAt
- function getResponseTime(incident) {
-   if (incident.status !== 'resolved') return null
-   const created = toDate(incident.createdAt)
-   const updated = toDate(incident.updatedAt)
-   if (!created || !updated) return null
-   const mins = (updated - created) / 60000
-   return mins > 0 ? mins : null
```

```
- }
+ // GET /api/analytics/all — full payload (used by Analytics.jsx)
+ router.get('/all', async (req, res, next) => {
+   try {
+     const data = await getAnalytics()
+     res.json(data)
+   } catch (error) {
+     next(error)
+   }
+ })

  // GET /api/analytics/summary
- // Returns: total incidents, resolved count, avg response time, this week count
  router.get('/summary', async (req, res, next) => {
    try {
-     const db = getDb()
-     const incidentsRef = db.collection('incidents')
-     const now = new Date()
-     const weekAgo = new Date(now.getTime() - 7 * 24 * 60 * 60 * 1000)
-
-     // Get all incidents
-     const allSnapshot = await incidentsRef.get()
-     const allIncidents = allSnapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }))
-
-     // Calculate metrics
-     const totalIncidents = allIncidents.length
-     const resolvedCount = allIncidents.filter(i => i.status === 'resolved').length
-
-     // Calculate average response time from createdAt vs updatedAt
-     const responseTimes = allIncidents.map(getResponseTime).filter(v => v !== null)
-     const avgResponseTime = responseTimes.length > 0
-       ? (responseTimes.reduce((sum, v) => sum + v, 0) / responseTimes.length).toFixed(1)
-       : 0
-
-     // This week incidents
```

```
48  -     const thisWeekIncidents = allIncidents.filter(i => {
49  -       const incidentDate = toDate(i.createdAt)
50  -       return incidentDate && incidentDate >= weekAgo
51  -     }).length
52  -
53  -     res.json({
54  -       totalIncidents,
55  -       resolvedCount,
56  -       avgResponseTime: parseFloat(avgResponseTime),
57  -       thisWeekIncidents,
58  -     })
    23 +     const data = await getAnalytics()
    24 +     res.json(data.summary)
59  25     } catch (error) {
60  26       next(error)
61  27     }
62  28   })
63  29
64  30   // GET /api/analytics/by-type
65  -  // Returns: incidents grouped by type with counts
66  31   router.get('/by-type', async (req, res, next) => {
67  32     try {
68  -     const db = getDb()
69  -     const snapshot = await db.collection('incidents').get()
70  -     const incidents = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }))
71  -
72  -     const typeCounts = {}
73  -     incidents.forEach(incident => {
74  -       const type = incident.type?.charAt(0).toUpperCase() + incident.type?.slice(1) || 'Unknown'
75  -       typeCounts[type] = (typeCounts[type] || 0) + 1
76  -     })
77  -
78  -     const data = Object.entries(typeCounts).map(([type, count]) => ({ type, count }))
79  -     res.json(data)
    33 +     const data = await getAnalytics()
```

```
+     res.json(data.incidentsByType)
    } catch (error) {
      next(error)
    }
  })

  // GET /api/analytics/status-breakdown
- // Returns: incidents grouped by status with counts
  router.get('/status-breakdown', async (req, res, next) => {
    try {
-     const db = getDb()
-     const snapshot = await db.collection('incidents').get()
-     const incidents = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }))
-
-     const statusMap = {
-       'resolved': 'Resolved',
-       'acknowledged': 'Acknowledged',
-       'triggered': 'Triggered',
-       'archived': 'Archived',
-     }
-
-     const statusCounts = {}
-     Object.values(statusMap).forEach(status => {
-       statusCounts[status] = 0
-     })
-
-     incidents.forEach(incident => {
-       const statusName = statusMap[incident.status] || 'Unknown'
-       statusCounts[statusName] = (statusCounts[statusName] || 0) + 1
-     })
-
-     const data = Object.entries(statusCounts).map(([name, value]) => ({ name, value }))
-     res.json(data)
+     const data = await getAnalytics()
+     res.json(data.statusBreakdown)
```

```
112  45     } catch (error) {
113  46       next(error)
114  47     }
115  48   })
116  49
117  50   // GET /api/analytics/by-location
118      - // Returns: incidents grouped by location with counts
119  51   router.get('/by-location', async (req, res, next) => {
120  52     try {
121      -     const db = getDb()
122      -     const snapshot = await db.collection('incidents').get()
123      -     const incidents = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }))
124      -
125      -     const locationCounts = {}
126      -     incidents.forEach(incident => {
127      -       const location = incident.location || 'Unknown'
128      -       locationCounts[location] = (locationCounts[location] || 0) + 1
129      -     })
130      -
131      -     const data = Object.entries(locationCounts).map(([location, count]) => ({ location, count }))
132      -     res.json(data)
     53  +     const data = await getAnalytics()
     54  +     res.json(data.locationData)
133  55     } catch (error) {
134  56       next(error)
135  57     }
136  58   })
137  59
138  60   // GET /api/analytics/this-week
139      - // Returns: incidents grouped by day of the week
140  61   router.get('/this-week', async (req, res, next) => {
141  62     try {
142      -     const db = getDb()
143      -     const snapshot = await db.collection('incidents').get()
144      -     const incidents = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }))
```

```
145 -
146 -     const now = new Date()
147 -     // Start of current week — Monday 00:00:00 UTC
148 -     const startOfWeek = new Date(now)
149 -     const day = startOfWeek.getUTCDay()
150 -     const diffToMonday = (day === 0 ? -6 : 1 - day)
151 -     startOfWeek.setUTCDate(startOfWeek.getUTCDate() + diffToMonday)
152 -     startOfWeek.setUTCHours(0, 0, 0, 0)
153 -
154 -     const days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
155 -     const dayMap = { 1: 'Mon', 2: 'Tue', 3: 'Wed', 4: 'Thu', 5: 'Fri', 6: 'Sat', 0: 'Sun' }
156 -     const dayCounts = {}
157 -     days.forEach(d => { dayCounts[d] = 0 })
158 -
159 -     incidents.forEach(incident => {
160 -       const incidentDate = toDate(incident.createdAt)
161 -       if (incidentDate && incidentDate >= startOfWeek && incidentDate <= now) {
162 -         const dayName = dayMap[incidentDate.getUTCDay()]
163 -         dayCounts[dayName]++
164 -       }
165 -     })
166 -
167 -     const data = days.map(d => ({ day: d, incidents: dayCounts[d] }))
168 -     res.json(data)
    +     const data = await getAnalytics()
    +     res.json(data.incidentsByDay)
169       } catch (error) {
170         next(error)
171       }
172     })
173
174     // GET /api/analytics/response-time-trend
175 - // Returns: average response time grouped by week
176     router.get('/response-time-trend', async (req, res, next) => {
177       try {
```

```
-     const db = getDb()
-     const snapshot = await db.collection('incidents').get()
-     const incidents = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }))
-
-     const now = new Date()
-     const weekData = {}
-
-     for (let i = 0; i < 4; i++) {
-       weekData[`Week ${i + 1}`] = { total: 0, count: 0 }
-     }
-
-     incidents.forEach(incident => {
-       const incidentDate = toDate(incident.createdAt)
-       const weekNumber = Math.floor((now - incidentDate) / (7 * 24 * 60 * 60 * 1000)) + 1
-
-       if (weekNumber >= 1 && weekNumber <= 4) {
-         const weekKey = `Week ${5 - weekNumber}`
-         if (weekData[weekKey]) {
-           const rt = getResponseTime(incident)
-           if (rt !== null) { weekData[weekKey].total += rt; weekData[weekKey].count++ }
-         }
-       }
-     })
-
-     const data = Object.entries(weekData).map(([week, data]) => ({
-       week,
-       avgMinutes: data.count > 0 ? (data.total / data.count).toFixed(1) : 0,
-     }))
-
-     res.json(data)
-   } catch (error) {
-     next(error)
-   }
- })
-
```

```
- // GET /api/analytics/all
- // Returns: all analytics data in one response
- router.get('/all', async (req, res, next) => {
-   try {
-     const db = getDb()
-     const snapshot = await db.collection('incidents').get()
-     const incidents = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }))
-
-     const now = new Date()
-     const weekAgo = new Date(now.getTime() - 7 * 24 * 60 * 60 * 1000)
-
-     // Summary metrics
-     const totalIncidents = incidents.length
-     const resolvedCount = incidents.filter(i => i.status === 'resolved').length
-     const allResponseTimes = incidents.map(getResponseTime).filter(v => v !== null)
-     const avgResponseTime = allResponseTimes.length > 0
-       ? (allResponseTimes.reduce((sum, v) => sum + v, 0) / allResponseTimes.length).toFixed(1)
-       : 0
-     const thisWeekIncidents = incidents.filter(i => {
-       const incidentDate = toDate(i.createdAt)
-       return incidentDate && incidentDate >= weekAgo
-     }).length
-
-     // By type
-     const typeCounts = {}
-     incidents.forEach(incident => {
-       const type = incident.type?.charAt(0).toUpperCase() + incident.type?.slice(1) || 'Unknown'
-       typeCounts[type] = (typeCounts[type] || 0) + 1
-     })
-     const incidentsByType = Object.entries(typeCounts).map(([type, count]) => ({ type, count }))
-
-     // Status breakdown
-     const statusMap = {
-       'resolved': 'Resolved',
-       'acknowledged': 'Acknowledged',
```

```
-       'triggered': 'Triggered',
-       'archived': 'Archived',
-     }
-     const statusCounts = {}
-     Object.values(statusMap).forEach(status => {
-       statusCounts[status] = 0
-     })
-     incidents.forEach(incident => {
-       const statusName = statusMap[incident.status] || 'Unknown'
-       statusCounts[statusName] = (statusCounts[statusName] || 0) + 1
-     })
-     const statusBreakdown = Object.entries(statusCounts).map(([name, value]) => ({ name, value }))
-
-     // By location
-     const locationCounts = {}
-     incidents.forEach(incident => {
-       const location = incident.location || 'Unknown'
-       locationCounts[location] = (locationCounts[location] || 0) + 1
-     })
-     const locationData = Object.entries(locationCounts).map(([location, count]) => ({ location, count }))
-
-     // This week by day — from Monday 00:00 UTC to now
-     const startOfWeek = new Date(now)
-     const dowOffset = (startOfWeek.getUTCDay() === 0 ? -6 : 1 - startOfWeek.getUTCDay())
-     startOfWeek.setUTCDate(startOfWeek.getUTCDate() + dowOffset)
-     startOfWeek.setUTCHours(0, 0, 0, 0)
-
-     const days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
-     const dayMap = { 1: 'Mon', 2: 'Tue', 3: 'Wed', 4: 'Thu', 5: 'Fri', 6: 'Sat', 0: 'Sun' }
-     const dayCounts = {}
-     days.forEach(d => { dayCounts[d] = 0 })
-     incidents.forEach(incident => {
-       const incidentDate = toDate(incident.createdAt)
-       if (incidentDate && incidentDate >= startOfWeek) {
-         dayCounts[dayMap[incidentDate.getUTCDay()]]++
```

```
-       }
-     })
-     const incidentsByDay = days.map(d => ({ day: d, incidents: dayCounts[d] }))
-
-     // Response time trend
-     const weekData = {}
-     for (let i = 0; i < 4; i++) {
-       weekData[`Week ${i + 1}`] = { total: 0, count: 0 }
-     }
-     incidents.forEach(incident => {
-       const incidentDate = toDate(incident.createdAt)
-       const weekNumber = Math.floor((now - incidentDate) / (7 * 24 * 60 * 60 * 1000)) + 1
-       if (weekNumber >= 1 && weekNumber <= 4) {
-         const weekKey = `Week ${5 - weekNumber}`
-         if (weekData[weekKey]) {
-           const rt = getResponseTime(incident)
-           if (rt !== null) { weekData[weekKey].total += rt; weekData[weekKey].count++ }
-         }
-       }
-     })
-     const responseTimeData = Object.entries(weekData).map(([week, data]) => ({
-       week,
-       avgMinutes: data.count > 0 ? parseFloat((data.total / data.count).toFixed(1)) : 0,
-     }))
-
-     res.json({
-       summary: {
-         totalIncidents,
-         resolvedCount,
-         avgResponseTime: parseFloat(avgResponseTime),
-         thisWeekIncidents,
-       },
-       incidentsByType,
-       statusBreakdown,
-       locationData,
```

```
318 -       incidentsByDay,
319 -       responseTimeData,
320 -     })
    + 73  const data = await getAnalytics()
    + 74  res.json(data.responseTimeData)
321   75    } catch (error) {
322   76      next(error)
323   77    }
```

3 server/src/routes/incidents.js

```
...  ...  @@ -1,5 +1,6 @@
1    1    const express = require('express')
2    2    const { docToObject, formatTimestamp, getDb, snapshotToArray } = require('../db/firebase')
     3  + const { invalidateAnalyticsCache } = require('../analyticsCache')
3    4
4    5    const router = express.Router()
5    6
92   93
93   94      const doc = await docRef.get()
94   95      const incident = { id: doc.id, ...doc.data() }
     96  +   invalidateAnalyticsCache()
95   97      res.status(201).json(toIncidentResponse(incident))
96   98    } catch (error) {
97   99      next(error)
103  105     const { status } = req.body
104  106     const now = new Date().toISOString()
105  107     await getDb().collection('incidents').doc(req.params.id).update({ status, updatedAt: now })
     108 +   invalidateAnalyticsCache()
106  109     res.json({ success: true })
107  110   } catch (error) {
108  111     next(error)
```